

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №3

по дисциплине: "Логика и основы алгоритмизации в инженерных задачах"

на тему: "Динамические списки"

Выполнил:

Студент группы 24ВВВ3

Макунькин А.М.

Мелюшев М.М.

Принял:

к.н., доцент, Юрова О. В.

к.т.н., Деев М.В.

Пенза 2025

Цель

Изучение динамических списков.

Лабораторное задание

Задание:

а. Реализовать приоритетную очередь, путём добавления элемента в список в соответствии с приоритетом объекта (т.е. объект с большим приоритетом становится перед объектом с меньшим приоритетом).

б. На основе приведенного кода реализуйте структуру данных Очередь.

с. На основе приведенного кода реализуйте структуру данных Стек.

Результат программы

```
D:\pr\Lab3L\x64\Debug\Lab3L.exe
Лабораторная работа №3: Динамические списки

=== МЕНЮ ===
1. Приоритетная очередь
2. Очередь (FIFO)
3. Стек (LIFO)
0. Выход
Выберите структуру: 1

=== ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ===
1. Добавить элемент
2. Удалить элемент по названию
3. Просмотреть очередь
0. Назад
Выберите действие: 1
Введите название объекта: легко
Введите приоритет (целое число): 1

=== ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ===
1. Добавить элемент
2. Удалить элемент по названию
3. Просмотреть очередь
0. Назад
Выберите действие: 1
Введите название объекта: средне
Введите приоритет (целое число): 3

=== ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ===
1. Добавить элемент
2. Удалить элемент по названию
3. Просмотреть очередь
0. Назад
Выберите действие: 1
Введите название объекта: сложно
Введите приоритет (целое число): 6

=== ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ===
1. Добавить элемент
2. Удалить элемент по названию
3. Просмотреть очередь
0. Назад
Выберите действие: 3
Содержимое приоритетной очереди:
Элемент: сложно (приоритет: 6)
Элемент: средне (приоритет: 3)
Элемент: легко (приоритет: 1)

=== ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ===
1. Добавить элемент
2. Удалить элемент по названию
3. Просмотреть очередь
```

Рисунок 1 – приоритетная очередь

```
D:\pr\Lab3L\x64\Debug\Lab3L.exe
Лабораторная работа №3: Динамические списки

=== МЕНЮ ===
1. Приоритетная очередь
2. Очередь (FIFO)
3. Стек (LIFO)
0. Выход
Выберите структуру: 2

=== ОЧЕРЕДЬ (FIFO) ===
1. Добавить в очередь
2. Извлечь из очереди
3. Просмотреть очередь
0. Назад
Выберите действие: 1
Введите название объекта: легко
Элемент добавлен в очередь: легко

=== ОЧЕРЕДЬ (FIFO) ===
1. Добавить в очередь
2. Извлечь из очереди
3. Просмотреть очередь
0. Назад
Выберите действие: 1
Введите название объекта: средне
Элемент добавлен в очередь: средне

=== ОЧЕРЕДЬ (FIFO) ===
1. Добавить в очередь
2. Извлечь из очереди
3. Просмотреть очередь
0. Назад
Выберите действие: 1
Введите название объекта: сложно
Элемент добавлен в очередь: сложно

=== ОЧЕРЕДЬ (FIFO) ===
1. Добавить в очередь
2. Извлечь из очереди
3. Просмотреть очередь
0. Назад
Выберите действие: 3
Содержимое списка:
Элемент: легко
Элемент: средне
Элемент: сложно

=== ОЧЕРЕДЬ (FIFO) ===
1. Добавить в очередь
2. Извлечь из очереди
3. Просмотреть очередь
0. Назад
Выберите действие:
```

Рисунок 2 – список «Очередь»

cs D:\pr\Lab3L\x64\Debug\Lab3L.exe

Лабораторная работа №3: Динамические списки

=== МЕНЮ ===

1. Приоритетная очередь
2. Очередь (FIFO)
3. Стек (LIFO)
0. Выход

Выберите структуру: 3

=== СТЕК (LIFO) ===

1. Добавить в стек (push)
2. Извлечь из стека (pop)
3. Просмотреть стек
0. Назад

Выберите действие: 1

Введите название объекта: легко

Элемент добавлен в стек: легко

=== СТЕК (LIFO) ===

1. Добавить в стек (push)
2. Извлечь из стека (pop)
3. Просмотреть стек
0. Назад

Выберите действие: 1

Введите название объекта: средне

Элемент добавлен в стек: средне

=== СТЕК (LIFO) ===

1. Добавить в стек (push)
2. Извлечь из стека (pop)
3. Просмотреть стек
0. Назад

Выберите действие: 1

Введите название объекта: сложно

Элемент добавлен в стек: сложно

=== СТЕК (LIFO) ===

1. Добавить в стек (push)
2. Извлечь из стека (pop)
3. Просмотреть стек
0. Назад

Выберите действие: 3

Содержимое списка:

Элемент: сложно

Элемент: средне

Элемент: легко

=== СТЕК (LIFO) ===

1. Добавить в стек (push)
2. Извлечь из стека (pop)
3. Просмотреть стек
0. Назад

Выберите действие:

Рисунок 3 – список «Стек»

Листинг

Файл 1-Lab3L.c

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <locale.h>
#include <windows.h>

#define MAX_INPUT_LENGTH 255

struct node
{
    char inf[MAX_INPUT_LENGTH + 1];
    struct node* next;
};

struct pnode
{
    char inf[MAX_INPUT_LENGTH + 1];
    int priority;
    struct pnode* next;
};

struct node* head = NULL, * last = NULL;
struct node* stack_top = NULL;
struct pnode* pqueue_head = NULL;

struct node* get_struct(void)
{
    struct node* p = NULL;
    char s[MAX_INPUT_LENGTH + 1];

    if ((p = (struct node*)malloc(sizeof(struct node))) == NULL)
    {
        printf("Ошибка при распределении памяти\n");
        exit(1);
    }

    printf("Введите название объекта: ");
    scanf("%255s", s);

    if (*s == 0)
    {
        printf("Запись не была произведена\n");
        free(p);
        return NULL;
    }

    strcpy(p->inf, s);
    p->next = NULL;

    return p;
}

struct pnode* get_pstruct(void)
{
    struct pnode* p = NULL;
    char s[MAX_INPUT_LENGTH + 1];
    int priority;

    if ((p = (struct pnode*)malloc(sizeof(struct pnode))) == NULL)
```

```

    {
        printf("Ошибка при распределении памяти\n");
        exit(1);
    }

    printf("Введите название объекта: ");
    scanf("%255s", s);
    printf("Введите приоритет (целое число): ");
    scanf("%d", &priority);

    if (*s == 0)
    {
        printf("Запись не была произведена\n");
        free(p);
        return NULL;
    }

    strcpy(p->inf, s);
    p->priority = priority;
    p->next = NULL;

    return p;
}

// Функция проверки наличия элемента в FIFO-очереди
int is_word_in_queue(const char* word)
{
    struct node* current = head;
    while (current != NULL)
    {
        if (strcmp(current->inf, word) == 0)
        {
            return 1; // Слово найдено
        }
        current = current->next;
    }
    return 0; // Слово не найдено
}

void review(struct node* head)
{
    struct node* struc = head;

    if (head == NULL)
    {
        printf("Список пуст\n");
        return;
    }

    printf("Содержимое списка:\n");
    while (struc)
    {
        printf("Элемент: %s\n", struc->inf);
        struc = struc->next;
    }
}

void pqueue_review(void)
{
    struct pnode* struc = pqueue_head;

    if (pqueue_head == NULL)
    {
        printf("Приоритетная очередь пуста\n");
        return;
    }
}

```

```

    }

    printf("Содержимое приоритетной очереди:\n");
    while (struc != NULL)
    {
        printf("Элемент: %s (приоритет: %d)\n", struc->inf, struc->priority);
        struc = struc->next;
    }
}

// ПРИОРИТЕТНАЯ ОЧЕРЕДЬ
void pqueue_store(void)
{
    struct pnode* p = get_pstruct();
    if (p == NULL) return;

    if (pqueue_head == NULL || p->priority > pqueue_head->priority)
    {
        p->next = pqueue_head;
        pqueue_head = p;
        return;
    }

    struct pnode* current = pqueue_head;
    while (current->next != NULL && current->next->priority >= p->priority)
    {
        current = current->next;
    }

    p->next = current->next;
    current->next = p;
}

void pqueue_pop(void)
{
    if (pqueue_head == NULL)
    {
        printf("Приоритетная очередь пуста\n");
        return;
    }

    char search[MAX_INPUT_LENGTH + 1];
    printf("Введите название элемента для удаления: ");
    scanf("%255s", search);

    struct pnode* current = pqueue_head;
    struct pnode* prev = NULL;
    int deleted = 0;

    while (current != NULL)
    {
        if (strcmp(current->inf, search) == 0)
        {
            struct pnode* to_delete = current;

            if (prev == NULL)
            {
                pqueue_head = current->next;
            }
            else
            {
                prev->next = current->next;
            }

            current = current->next;
        }
    }
}

```

```

        printf("Удален элемент: %s (приоритет: %d)\n", to_delete->inf,
to_delete->priority);
        free(to_delete);
        deleted++;
    }
    else
    {
        prev = current;
        current = current->next;
    }
}

if (deleted == 0)
{
    printf("Элемент не найден\n");
}
}

// ОЧЕРЕДЬ FIFO - ТОЛЬКО ЗДЕСЬ ЗАПРЕЩЕНЫ ПОВТОРЫ
void queue_store(void)
{
    struct node* p = get_struct();
    if (p == NULL) return;

    // Проверяем, нет ли уже такого слова в очереди
    if (is_word_in_queue(p->inf))
    {
        printf("Ошибка: слово '%s' уже есть в очереди. Повторы запрещены.\n", p-
>inf);
        free(p);
        return;
    }

    if (head == NULL)
    {
        head = p;
        last = p;
    }
    else
    {
        last->next = p;
        last = p;
    }
    printf("Элемент добавлен в очередь: %s\n", p->inf);
}

void queue_pop(void)
{
    if (head == NULL)
    {
        printf("Очередь пуста\n");
        return;
    }

    struct node* temp = head;
    head = head->next;

    if (head == NULL)
    {
        last = NULL;
    }

    printf("Извлечен из очереди: %s\n", temp->inf);
    free(temp);
}

```



```

// СТЕК - здесь повторы разрешены
void stack_push(void)
{
    struct node* p = get_struct();
    if (p == NULL) return;

    p->next = stack_top;
    stack_top = p;
    printf("Элемент добавлен в стек: %s\n", p->inf);
}

void stack_pop(void)
{
    if (stack_top == NULL)
    {
        printf("Стек пуст\n");
        return;
    }

    struct node* temp = stack_top;
    stack_top = stack_top->next;
    printf("Извлечен из стека: %s\n", temp->inf);
    free(temp);
}

void stack_review(void)
{
    review(stack_top);
}

void print_menu(void)
{
    printf("\n=== МЕНЮ ===\n");
    printf("1. Приоритетная очередь\n");
    printf("2. Очередь (FIFO)\n");
    printf("3. Стек (LIFO)\n");
    printf("0. Выход\n");
    printf("Выберите структуру: ");
}

void pqueue_menu(void)
{
    int choice;
    do {
        printf("\n=== ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ===\n");
        printf("1. Добавить элемент\n");
        printf("2. Удалить элемент по названию\n");
        printf("3. Просмотреть очередь\n");
        printf("0. Назад\n");
        printf("Выберите действие: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: pqueue_store(); break;
            case 2: pqueue_pop(); break;
            case 3: pqueue_review(); break;
            case 0: break;
            default: printf("Неверный выбор!\n");
        }
    } while (choice != 0);
}

void queue_menu(void)
{

```

```

int choice;
do {
    printf("\n=== ОЧЕРЕДЬ (FIFO) ===\n");
    printf("1. Добавить в очередь\n");
    printf("2. Извлечь из очереди\n");
    printf("3. Просмотреть очередь\n");
    printf("0. Назад\n");
    printf("Выберите действие: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1: queue_store(); break;
        case 2: queue_pop(); break;
        case 3: review(head); break;
        case 0: break;
        default: printf("Неверный выбор!\n");
    }
} while (choice != 0);
}

void stack_menu(void)
{
    int choice;
    do {
        printf("\n=== СТЕК (LIFO) ===\n");
        printf("1. Добавить в стек (push)\n");
        printf("2. Извлечь из стека (pop)\n");
        printf("3. Просмотреть стек\n");
        printf("0. Назад\n");
        printf("Выберите действие: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: stack_push(); break;
            case 2: stack_pop(); break;
            case 3: stack_review(); break;
            case 0: break;
            default: printf("Неверный выбор!\n");
        }
    } while (choice != 0);
}

int main(void)
{
    SetConsoleOutputCP(1251);
    SetConsoleCP(1251);
    setlocale(LC_ALL, "Russian");
    int main_choice;

    printf("Лабораторная работа №3: Динамические списки\n");

    do {
        print_menu();
        scanf("%d", &main_choice);

        switch (main_choice) {
            case 1: pqueue_menu(); break;
            case 2: queue_menu(); break;
            case 3: stack_menu(); break;
            case 0: printf("Выход...\n"); break;
            default: printf("Неверный выбор!\n");
        }
    } while (main_choice != 0);

    return 0;
}

```

Вывод

В ходе выполнения лабораторной работы были разработаны программы для выполнения заданий Лабораторной работы №3. В процессе выполнения работы были использованы знания о динамических списках.

Ссылка на удаленный репозиторий GitHub:

<https://github.com/LareklAREK/LiOAiIZ-2OURSE-.git>

<https://github.com/Motorrr/LiOAvIZ.git>